

Smart Parking – Cheat Sheet

1. Datenbank

`parking_sessions`

Speichert die Parkvorgänge der Fahrzeuge.

Wichtige Felder:

Feld	Bedeutung
<code>id</code>	Session-ID
<code>plate</code>	Kennzeichen
<code>entry_time</code>	Zeitpunkt der Einfahrt
<code>exit_time</code>	Zeitpunkt der Ausfahrt
<code>payment_time</code>	Zeitpunkt der Zahlung
<code>exit_valid_until</code>	Ausfahrt nach Zahlung bis zu diesem Zeitpunkt erlaubt
<code>amount_due</code>	Zu zahlender Betrag
<code>amount_paid</code>	Bereits gezahlter Betrag

Wichtig:

`exit_time = NULL` bedeutet, dass das Fahrzeug noch im Parkhaus ist.

`event_logs`

Protokolliert wichtige Ereignisse im System.

Wichtige Felder:

Feld	Bedeutung
<code>timestamp</code>	Zeitpunkt
<code>eventtype</code>	Art des Events
<code>source</code>	Quelle, z. B. Kamera oder Sensor
<code>message</code>	Beschreibung
<code>parking_session_id</code>	Optionale Zuordnung zu einer Parksession

Beispiele für Events:

- ENTRY_PLATE_DETECTED
- EXIT_PLATE_DETECTED
- ENTRY_BARRIER_OPENED
- EXIT_BARRIER_OPENED
- EXIT_BARRIER_DENIED
- PAYMENT_SUCCESS
- RFID_ADMIN_RELEASE
- UNKNOWN_RFID
- PARKING_SPOT_UPDATED
- LIGHT_TURNED_ON
- LIGHT_TURNED_OFF
- SENSOR_ERROR

parking_spots

Speichert den aktuellen Zustand der Stellplätze.

Feld	Bedeutung
id	Stellplatz-ID
floor	Etage
spot_number	Platznummer
sensor_id	Zugeordneter Sensor
occupied	0 = frei, 1 = belegt
last_updated	Letzte Aktualisierung

user_admins

Speichert erlaubte RFID-Chips für manuelle Ausfahrtsfreigaben.

Feld	Bedeutung
name	Name des Admins
uid	RFID-UID
active	RFID aktiv oder deaktiviert

rfid_admin_releases

Protokolliert manuelle Ausfahrtsfreigaben.

Feld	Bedeutung
<code>admin_id</code>	Verwendeter Admin
<code>rfid_uid</code>	Verwendete RFID-UID
<code>parking_session_id</code>	Betroffene Parksession
<code>release_time</code>	Zeitpunkt der Freigabe

2. MQTT-Topics

Sensoren → Backend

`einfahren/plate`

Die Einfahrtskamera sendet ein erkanntes Kennzeichen.

```
{
  "plate": "RAB1234"
}
```

Das Ergebnis kommt über:

`einfahren/status`

`ausfahren/plate`

Die Ausfahrtskamera sendet das Kennzeichen des Fahrzeugs.

```
{
  "plate": "RAB1234"
}
```

Das Ergebnis kommt über:

`ausfahren/status`

`ausfahren/rfid`

RFID-Scan für eine manuelle Freigabe nach verweigerter Ausfahrt.

```
{
  "uid": "A1-B2-C3-D4"
}
```

Der Scan ist nur während des aktiven RFID-Zeitfensters relevant.

Das Ergebnis kommt ebenfalls über:

```
ausfahren/status
```

```
parking/spot/status
```

Ein Stellplatzsensor meldet seinen aktuellen Zustand.

```
{
  "id": 1,
  "status": "occupied"
}
```

Erlaubte Statuswerte:

```
occupied
free
```

```
building/light/status
```

Ein Gerät meldet den Lichtstatus einer Etage.

```
{
  "floor": 1,
  "status": "on"
}
```

Erlaubte Werte:

- floor: 1 oder 2
- status: on oder off

Die Zeitsteuerung des Lichts liegt auf dem Gerät.

Backend → Aktoren

einfahren/status

Antwort an die Einfahrtsschranke und das Display.

Einfahrt erlaubt

```
{
  "status": "allowed",
  "display_text": "Willkommen RAB1234"
}
```

Einfahrt verweigert

```
{
  "status": "denied",
  "display_text": "Bitte bei der Parkhausleitung melden"
}
```

Parkhaus voll

```
{
  "status": "denied",
  "display_text": "Parkhaus voll - keine freien Plaetze"
}
```

ausfahren/status

Antwort an die Ausfahrtsschranke und das Display.

Ausfahrt erlaubt

```
{
  "status": "allowed",
  "display_text": "Gute Fahrt"
}
```

Ausfahrt verweigert

```
{
  "status": "denied",
  "display_text": "RAB1234 nicht erlaubt - RFID-Chip halten oder bezahlen"
}
```

```
(30s)"  
}
```

Nach dieser Meldung kann innerhalb von 30 Sekunden ein RFID-Chip gescannt werden.

RFID-Freigabe

```
{  
  "status": "allowed",  
  "display_text": "RFID-Freigabe erteilt"  
}
```

Unbekannte RFID

```
{  
  "status": "denied",  
  "display_text": "RFID unbekannt - keine Freigabe"  
}
```

Einheitliches Aktor-Format:

```
{  
  "status": "allowed oder denied",  
  "display_text": "Text fuer das Display"  
}
```

Der ESP verarbeitet Motor, LEDs und Display selbst.

3. REST-Schnittstellen

Admin-Dashboard

Event-Logs laden

```
GET /api/admin/event-logs
```

Mögliche Filter:

```
limit  
offset  
eventtype  
plate
```

```
from  
to
```

Beispiel:

```
GET /api/admin/event-logs?limit=50&eventtype=EXIT_BARRIER_DENIED
```

Offene Parkvorgänge

```
GET /api/admin/parking-sessions/open
```

Einzelne Session

```
GET /api/admin/parking-sessions/:sessionId
```

Session bearbeiten

```
PATCH /api/admin/parking-sessions/:sessionId
```

```
{  
  "amount_paid": 3.0,  
  "payment_time": "2026-06-11T16:20:00Z",  
  "exit_valid_until": "2026-06-11T16:35:00Z"  
}
```

RFID-Admins laden

```
GET /api/admin/rfid-admins
```

RFID-Admin anlegen

```
POST /api/admin/rfid-admins
```

```
{  
  "name": "Test Admin",  
  "uid": "A1-B2-C3-D4",  
  "active": true  
}
```

RFID-Admin bearbeiten

```
PATCH /api/admin/rfid-admins/:uid
```

```
{
  "active": false
}
```

Manuelle RFID-Freigaben laden

```
GET /api/admin/rfid-releases
```

Zahlungs-Dashboard

Offene Sessions laden

```
GET /api/payments/open-sessions
```

Beispiel:

```
[
  {
    "session_id": 12,
    "plate": "RAB1234",
    "entry_time": "2026-06-11T14:10:00Z",
    "amount_due": 3.0,
    "amount_paid": 1.0,
    "remaining_amount": 2.0
  }
]
```

Session über Kennzeichen suchen

```
GET /api/payments/session/by-plate/:plate
```

Beispiel:

```
{
  "session_id": 12,
  "plate": "RAB1234",
  "entry_time": "2026-06-11T14:10:00Z",
  "amount_due": 3.0,
```

```
"amount_paid": 1.0,  
"remaining_amount": 2.0  
}
```

Zahlung eintragen

```
POST /api/payments/pay
```

```
{  
  "plate": "RAB1234",  
  "amount_paid_now": 2.0  
}
```

Antwort:

```
{  
  "success": true,  
  "plate": "RAB1234",  
  "session_id": 12,  
  "amount_due": 3.0,  
  "amount_paid_total": 3.0,  
  "remaining_amount": 0.0,  
  "exit_allowed": true  
}
```

Teilzahlungen sind möglich. Der eingezahlte Betrag wird immer über `amount_paid_now` mitgegeben.

Session direkt bearbeiten

```
PATCH /api/payments/session/:sessionId
```

```
{  
  "amount_paid": 2.0  
}
```

Map-Dashboard

Alle Stellplätze laden

```
GET /api/map/parking-spots
```

```
[
  {
    "id": 1,
    "floor": 1,
    "spot_number": 1,
    "occupied": 0
  },
  {
    "id": 2,
    "floor": 1,
    "spot_number": 2,
    "occupied": 1
  }
]
```

Übersicht laden

```
GET /api/map/overview
```

```
{
  "total_spots": 100,
  "free_spots": 42,
  "occupied_spots": 58
}
```

4. Wichtig für die Integration

- MQTT-Nachrichten werden als JSON gesendet.
- Kennzeichen werden über das Feld `plate` übertragen.
- RFID-Chips werden über das Feld `uid` übertragen.
- Aktor-Antworten enthalten immer `status` und `display_text`.
- `status` ist entweder `allowed` oder `denied`.
- Die ESPs steuern Schranke, LEDs und Display selbst.
- Das Map-Dashboard verwendet `0` für frei und `1` für belegt.
- Nach einer verweigerten Ausfahrt ist ein RFID-Scan für 30 Sekunden möglich.
- Nach einer erfolgreichen Zahlung ist die Ausfahrt für 15 Minuten gültig.
- Admin- und Map-Dashboard dürfen dieselben Map-Endpunkte verwenden.